# 1 Supervised Learning

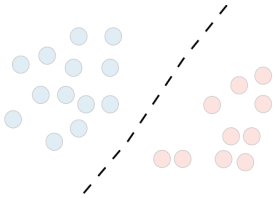
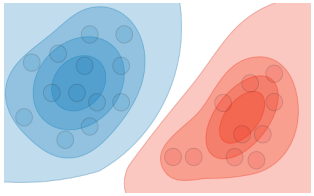
## 1.1 Introduction to Supervised Learning

Given a set of data points $\{x^{(1)},...,x^{(m)}\}$ associated to a set of outcomes $\{y^{(1)},...,y^{(m)}\}$, we want to build a classifier that learns how to predict $y$ from $x$.

❒ **Type of prediction** – The different types of predictive models are summed up in the table below:

| | Regression | Classifier |
|---|---|---|
| **Outcome** | Continuous | Class |
| **Examples** | Linear regression | Logistic regression, SVM, Naive Bayes |

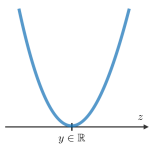
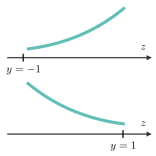
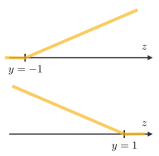
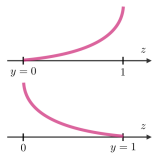
❒ **Type of model** – The different models are summed up in the table below:

| | Discriminative model | Generative model |
|---|---|---|
| **Goal** | Directly estimate $P(y\|x)$ | Estimate $P(x\|y)$ to deduce $P(y\|x)$ |
| **What's learned** | Decision boundary | Probability distributions of the data |
| **Illustration** |  |  |
| **Examples** | Regressions, SVMs | GDA, Naive Bayes |

## 1.2 Notations and general concepts

❒ **Hypothesis** – The hypothesis is noted $h_\theta$ and is the model that we choose. For a given input data $x^{(i)}$, the model prediction output is $h_\theta(x^{(i)})$.

❒ **Loss function** – A loss function is a function $L:(z,y)\in\mathbb{R}\times Y\longmapsto L(z,y)\in\mathbb{R}$ that takes as inputs the predicted value $z$ corresponding to the real data value $y$ and outputs how different they are. The common loss functions are summed up in the table below:

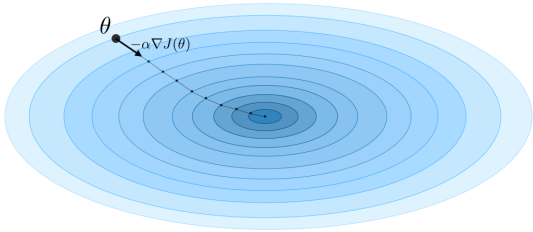
| Least squared | Logistic | Hinge | Cross-entropy |
|---|---|---|---|
| $\frac{1}{2}(y-z)^2$ | $\log(1+\exp(-yz))$ | $\max(0,1-yz)$ | $-\left[y\log(z)+(1-y)\log(1-z)\right]$ |
|  |  |  |  |
| Linear regression | Logistic regression | SVM | Neural Network |

❒ **Cost function** – The cost function $J$ is commonly used to assess the performance of a model, and is defined with the loss function $L$ as follows:

$$\boxed{J(\theta)=\sum_{i=1}^{m}L(h_\theta(x^{(i)}),y^{(i)})}$$

❒ **Gradient descent** – By noting $\alpha\in\mathbb{R}$ the learning rate, the update rule for gradient descent is expressed with the learning rate and the cost function $J$ as follows:

$$\boxed{\theta\longleftarrow\theta-\alpha\nabla J(\theta)}$$




*Remark: Stochastic gradient descent (SGD) is updating the parameter based on each training example, and batch gradient descent is on a batch of training examples.*

❒ **Likelihood** – The likelihood of a model $L(\theta)$ given parameters $\theta$ is used to find the optimal parameters $\theta$ through maximizing the likelihood. In practice, we use the log-likelihood $\ell(\theta)=\log(L(\theta))$ which is easier to optimize. We have:

$$\boxed{\theta^{\text{opt}}=\underset{\theta}{\arg\max}\ L(\theta)}$$

❒ **Newton's algorithm** – The Newton's algorithm is a numerical method that finds $\theta$ such that $\ell'(\theta)=0$. Its update rule is as follows:

$$\boxed{\theta\leftarrow\theta-\frac{\ell'(\theta)}{\ell''(\theta)}}$$

*Remark: the multidimensional generalization, also known as the Newton-Raphson method, has the following update rule:*

$$\theta\leftarrow\theta-\left(\nabla^2_\theta\ell(\theta)\right)^{-1}\nabla_\theta\ell(\theta)$$

## 1.3 Linear models

### 1.3.1 Linear regression

We assume here that $y|x;\theta\sim\mathcal{N}(\mu,\sigma^2)$

❒ **Normal equations** – By noting $X$ the matrix design, the value of $\theta$ that minimizes the cost function is a closed-form solution such that:

$$\boxed{\theta=(X^TX)^{-1}X^Ty}$$

❒ **LMS algorithm** – By noting $\alpha$ the learning rate, the update rule of the Least Mean Squares (LMS) algorithm for a training set of $m$ data points, which is also known as the Widrow-Hoff learning rule, is as follows:

$$\boxed{\forall j, \quad \theta_j \leftarrow \theta_j + \alpha \sum_{i=1}^{m} \left[y^{(i)} - h_\theta(x^{(i)})\right] x_j^{(i)}}$$

*Remark: the update rule is a particular case of the gradient ascent.*

❒ **LWR** – Locally Weighted Regression, also known as LWR, is a variant of linear regression that weights each training example in its cost function by $w^{(i)}(x)$, which is defined with parameter $\tau \in \mathbb{R}$ as:

$$\boxed{w^{(i)}(x) = \exp\left(-\frac{(x^{(i)} - x)^2}{2\tau^2}\right)}$$

### 1.3.2 Classification and logistic regression

❒ **Sigmoid function** – The sigmoid function $g$, also known as the logistic function, is defined as follows:

$$\forall z \in \mathbb{R}, \quad \boxed{g(z) = \frac{1}{1+e^{-z}} \in ]0,1[}$$

❒ **Logistic regression** – We assume here that $y|x;\theta \sim \text{Bernoulli}(\phi)$. We have the following form:

$$\boxed{\phi = p(y=1|x;\theta) = \frac{1}{1+\exp(-\theta^T x)} = g(\theta^T x)}$$

*Remark: there is no closed form solution for the case of logistic regressions.*

❒ **Softmax regression** – A softmax regression, also called a multiclass logistic regression, is used to generalize logistic regression when there are more than 2 outcome classes. By convention, we set $\theta_K = 0$, which makes the Bernoulli parameter $\phi_i$ of each class $i$ equal to:

$$\boxed{\phi_i = \frac{\exp(\theta_i^T x)}{\sum_{j=1}^{K} \exp(\theta_j^T x)}}$$

### 1.3.3 Generalized Linear Models

❒ **Exponential family** – A class of distributions is said to be in the exponential family if it can be written in terms of a natural parameter, also called the canonical parameter or link function, $\eta$, a sufficient statistic $T(y)$ and a log-partition function $a(\eta)$ as follows:

$$\boxed{p(y;\eta) = b(y)\exp(\eta T(y) - a(\eta))}$$

*Remark: we will often have $T(y) = y$. Also, $\exp(-a(\eta))$ can be seen as a normalization parameter that will make sure that the probabilities sum to one.*

Here are the most common exponential distributions summed up in the following table:

| Distribution | $\eta$ | $T(y)$ | $a(\eta)$ | $b(y)$ |
|---|---|---|---|---|
| Bernoulli | $\log\left(\frac{\phi}{1-\phi}\right)$ | $y$ | $\log(1+\exp(\eta))$ | 1 |
| Gaussian | $\mu$ | $y$ | $\frac{\eta^2}{2}$ | $\frac{1}{\sqrt{2\pi}}\exp\left(-\frac{y^2}{2}\right)$ |
| Poisson | $\log(\lambda)$ | $y$ | $e^\eta$ | $\frac{1}{y!}$ |
| Geometric | $\log(1-\phi)$ | $y$ | $\log\left(\frac{e^\eta}{1-e^\eta}\right)$ | 1 |

❒ **Assumptions of GLMs** – Generalized Linear Models (GLM) aim at predicting a random variable $y$ as a function fo $x \in \mathbb{R}^{n+1}$ and rely on the following 3 assumptions:

(1) $\boxed{y|x;\theta \sim \text{ExpFamily}(\eta)}$ (2) $\boxed{h_\theta(x) = E[y|x;\theta]}$ (3) $\boxed{\eta = \theta^T x}$

*Remark: ordinary least squares and logistic regression are special cases of generalized linear models.*

## 1.4 Support Vector Machines

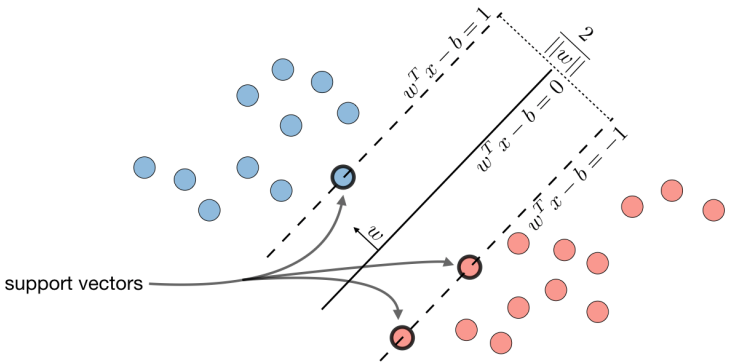
The goal of support vector machines is to find the line that maximizes the minimum distance to the line.

❒ **Optimal margin classifier** – The optimal margin classifier $h$ is such that:

$$\boxed{h(x) = \text{sign}(w^T x - b)}$$

where $(w,b) \in \mathbb{R}^n \times \mathbb{R}$ is the solution of the following optimization problem:

$$\boxed{\min \frac{1}{2}||w||^2} \quad \text{such that} \quad \boxed{y^{(i)}(w^T x^{(i)} - b) \geqslant 1}$$




*Remark: the line is defined as* $\boxed{w^T x - b = 0}$.

❒ **Hinge loss** – The hinge loss is used in the setting of SVMs and is defined as follows:

$$\boxed{L(z,y) = [1-yz]_+ = \max(0, 1-yz)}$$

# 2 Unsupervised Learning

## 2.1 Introduction to Unsupervised Learning

❒ **Motivation** – The goal of unsupervised learning is to find hidden patterns in unlabeled data $\{x^{(1)},...,x^{(m)}\}$.

❒ **Jensen's inequality** – Let $f$ be a convex function and $X$ a random variable. We have the following inequality:

$$\boxed{E[f(X)] \geqslant f(E[X])}$$

## 2.2 Clustering

### 2.2.1 Expectation-Maximization

❒ **Latent variables** – Latent variables are hidden/unobserved variables that make estimation problems difficult, and are often denoted $z$. Here are the most common settings where there are latent variables:

| Setting | Latent variable $z$ | $x\|z$ | Comments |
|---|---|---|---|
| Mixture of $k$ Gaussians | Multinomial($\phi$) | $\mathcal{N}(\mu_j,\Sigma_j)$ | $\mu_j \in \mathbb{R}^n, \phi \in \mathbb{R}^k$ |
| Factor analysis | $\mathcal{N}(0,I)$ | $\mathcal{N}(\mu+\Lambda z,\psi)$ | $\mu_j \in \mathbb{R}^n$ |

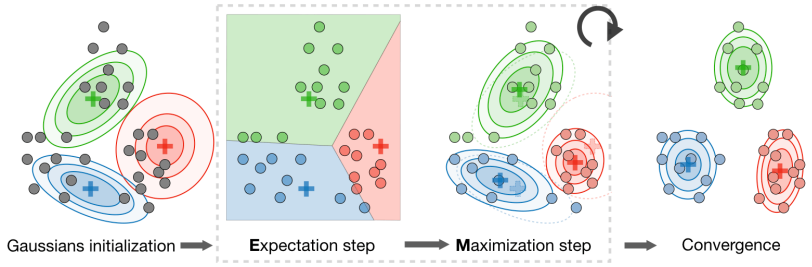
❒ **Algorithm** – The Expectation-Maximization (EM) algorithm gives an efficient method at estimating the parameter $\theta$ through maximum likelihood estimation by repeatedly constructing a lower-bound on the likelihood (E-step) and optimizing that lower bound (M-step) as follows:

- E-step: Evaluate the posterior probability $Q_i(z^{(i)})$ that each data point $x^{(i)}$ came from a particular cluster $z^{(i)}$ as follows:

$$\boxed{Q_i(z^{(i)}) = P(z^{(i)}|x^{(i)};\theta)}$$

- M-step: Use the posterior probabilities $Q_i(z^{(i)})$ as cluster specific weights on data points $x^{(i)}$ to separately re-estimate each cluster model as follows:

$$\boxed{\theta_i = \underset{\theta}{\text{argmax}} \sum_i \int_{z^{(i)}} Q_i(z^{(i)}) \log\left(\frac{P(x^{(i)},z^{(i)};\theta)}{Q_i(z^{(i)})}\right) dz^{(i)}}$$

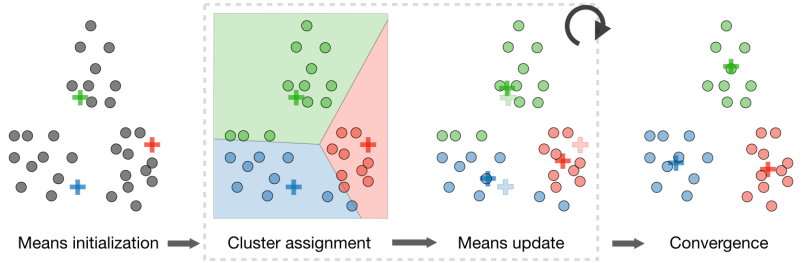



### 2.2.2 $k$-means clustering

We note $c^{(i)}$ the cluster of data point $i$ and $\mu_j$ the center of cluster $j$.

❒ **Algorithm** – After randomly initializing the cluster centroids $\mu_1,\mu_2,...,\mu_k \in \mathbb{R}^n$, the $k$-means algorithm repeats the following step until convergence:

$$\boxed{c^{(i)} = \underset{j}{\arg\min}||x^{(i)}-\mu_j||^2} \quad \text{and} \quad \boxed{\mu_j = \frac{\sum_{i=1}^m 1_{\{c^{(i)}=j\}} x^{(i)}}{\sum_{i=1}^m 1_{\{c^{(i)}=j\}}}}$$




❒ **Distortion function** – In order to see if the algorithm converges, we look at the distortion function defined as follows:

$$\boxed{J(c,\mu) = \sum_{i=1}^m ||x^{(i)} - \mu_{c^{(i)}}||^2}$$

### 2.2.3 Hierarchical clustering

❒ **Algorithm** – It is a clustering algorithm with an agglomerative hierarchical approach that build nested clusters in a successive manner.

❒ **Types** – There are different sorts of hierarchical clustering algorithms that aims at optimizing different objective functions, which is summed up in the table below:

| Ward linkage | Average linkage | Complete linkage |
|---|---|---|
| Minimize within cluster distance | Minimize average distance between cluster pairs | Minimize maximum distance of between cluster pairs |

### 2.2.4 Clustering assessment metrics

In an unsupervised learning setting, it is often hard to assess the performance of a model since we don't have the ground truth labels as was the case in the supervised learning setting.

❒ **Silhouette coefficient** – By noting $a$ and $b$ the mean distance between a sample and all other points in the same class, and between a sample and all other points in the next nearest cluster, the silhouette coefficient $s$ for a single sample is defined as follows:

$$\boxed{s = \frac{b-a}{\max(a,b)}}$$

❒ **Calinski-Harabaz index** – By noting $k$ the number of clusters, $B_k$ and $W_k$ the between and within-clustering dispersion matrices respectively defined as

$$B_k = \sum_{j=1}^{k} n_{c^{(i)}}(\mu_{c^{(i)}} - \mu)(\mu_{c^{(i)}} - \mu)^T, \qquad W_k = \sum_{i=1}^{m}(x^{(i)} - \mu_{c^{(i)}})(x^{(i)} - \mu_{c^{(i)}})^T$$

the Calinski-Harabaz index $s(k)$ indicates how well a clustering model defines its clusters, such that the higher the score, the more dense and well separated the clusters are. It is defined as follows:

$$\boxed{s(k) = \frac{\text{Tr}(B_k)}{\text{Tr}(W_k)} \times \frac{N-k}{k-1}}$$

## 2.3 Dimension reduction

### 2.3.1 Principal component analysis

It is a dimension reduction technique that finds the variance maximizing directions onto which to project the data.

❒ **Eigenvalue, eigenvector** – Given a matrix $A \in \mathbb{R}^{n\times n}$, $\lambda$ is said to be an eigenvalue of $A$ if there exists a vector $z \in \mathbb{R}^n \backslash \{0\}$, called eigenvector, such that we have:

$$\boxed{Az = \lambda z}$$

❒ **Spectral theorem** – Let $A \in \mathbb{R}^{n\times n}$. If $A$ is symmetric, then $A$ is diagonalizable by a real orthogonal matrix $U \in \mathbb{R}^{n\times n}$. By noting $\Lambda = \text{diag}(\lambda_1,...,\lambda_n)$, we have:

$$\boxed{\exists \Lambda \text{ diagonal}, \quad A = U\Lambda U^T}$$

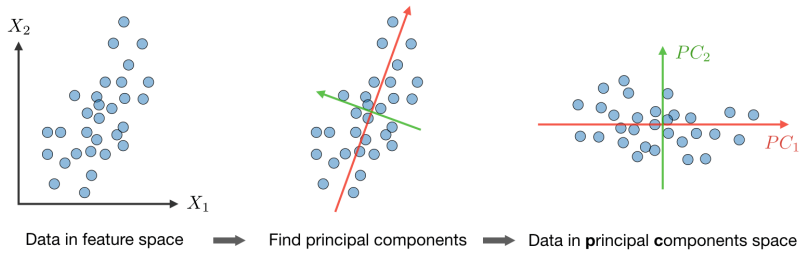
*Remark: the eigenvector associated with the largest eigenvalue is called principal eigenvector of matrix A.*

❒ **Algorithm** – The Principal Component Analysis (PCA) procedure is a dimension reduction technique that projects the data on $k$ dimensions by maximizing the variance of the data as follows:

- Step 1: Normalize the data to have a mean of 0 and standard deviation of 1.

$$\boxed{x_j^{(i)} \leftarrow \frac{x_j^{(i)} - \mu_j}{\sigma_j}} \quad \text{where} \quad \boxed{\mu_j = \frac{1}{m}\sum_{i=1}^{m} x_j^{(i)}} \quad \text{and} \quad \boxed{\sigma_j^2 = \frac{1}{m}\sum_{i=1}^{m}(x_j^{(i)} - \mu_j)^2}$$

- Step 2: Compute $\Sigma = \dfrac{1}{m}\displaystyle\sum_{i=1}^{m} x^{(i)}x^{(i)^T} \in \mathbb{R}^{n\times n}$, which is symmetric with real eigenvalues.
- Step 3: Compute $u_1, ..., u_k \in \mathbb{R}^n$ the $k$ orthogonal principal eigenvectors of $\Sigma$, i.e. the orthogonal eigenvectors of the $k$ largest eigenvalues.
- Step 4: Project the data on $\text{span}_{\mathbb{R}}(u_1,...,u_k)$. This procedure maximizes the variance among all $k$-dimensional spaces.




### 2.3.2 Independent component analysis

It is a technique meant to find the underlying generating sources.

❒ **Assumptions** – We assume that our data $x$ has been generated by the $n$-dimensional source vector $s = (s_1,...,s_n)$, where $s_i$ are independent random variables, via a mixing and non-singular matrix $A$ as follows:

$$\boxed{x = As}$$

The goal is to find the unmixing matrix $W = A^{-1}$ by an update rule.

❒ **Bell and Sejnowski ICA algorithm** – This algorithm finds the unmixing matrix $W$ by following the steps below:

- Write the probability of $x = As = W^{-1}s$ as:

$$p(x) = \prod_{i=1}^{n} p_s(w_i^T x) \cdot |W|$$

- Write the log likelihood given our training data $\{x^{(i)}, i \in [\![1,m]\!]\}$ and by noting $g$ the sigmoid function as:

$$l(W) = \sum_{i=1}^{m}\left(\sum_{j=1}^{n} \log\left(g'(w_j^T x^{(i)})\right) + \log|W|\right)$$

Therefore, the stochastic gradient ascent learning rule is such that for each training example $x^{(i)}$, we update $W$ as follows:

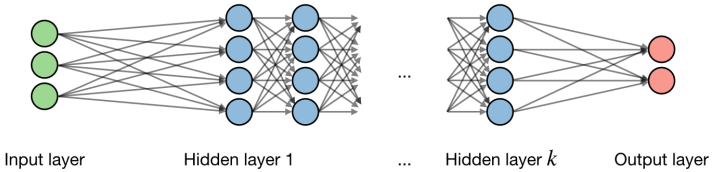
$$\boxed{W \longleftarrow W + \alpha\left(\begin{pmatrix} 1 - 2g(w_1^T x^{(i)}) \\ 1 - 2g(w_2^T x^{(i)}) \\ \vdots \\ 1 - 2g(w_n^T x^{(i)}) \end{pmatrix} x^{(i)^T} + (W^T)^{-1}\right)}$$

# 3 Deep Learning

## 3.1 Neural Networks

Neural networks are a class of models that are built with layers. Commonly used types of neural networks include convolutional and recurrent neural networks.

❒ **Architecture** – The vocabulary around neural networks architectures is described in the figure below:

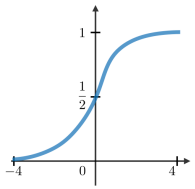
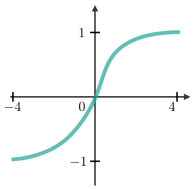
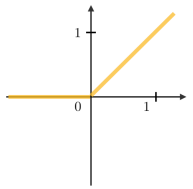
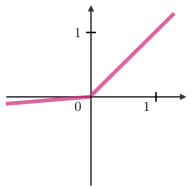



By noting $i$ the $i^{th}$ layer of the network and $j$ the $j^{th}$ hidden unit of the layer, we have:

$$\boxed{z_j^{[i]} = {w_j^{[i]}}^T x + b_j^{[i]}}$$

where we note $w$, $b$, $z$ the weight, bias and output respectively.

❒ **Activation function** – Activation functions are used at the end of a hidden unit to introduce non-linear complexities to the model. Here are the most common ones:

| Sigmoid | Tanh | ReLU | Leaky ReLU |
|---|---|---|---|
| $g(z) = \dfrac{1}{1+e^{-z}}$ | $g(z) = \dfrac{e^z - e^{-z}}{e^z + e^{-z}}$ | $g(z) = \max(0,z)$ | $g(z) = \max(\epsilon z, z)$ with $\epsilon \ll 1$ |
|  |  |  |  |

❒ **Cross-entropy loss** – In the context of neural networks, the cross-entropy loss $L(z,y)$ is commonly used and is defined as follows:

$$\boxed{L(z,y) = -\Big[y\log(z) + (1-y)\log(1-z)\Big]}$$

❒ **Learning rate** – The learning rate, often noted $\eta$, indicates at which pace the weights get updated. This can be fixed or adaptively changed. The current most popular method is called Adam, which is a method that adapts the learning rate.

❒ **Backpropagation** – Backpropagation is a method to update the weights in the neural network by taking into account the actual output and the desired output. The derivative with respect to weight $w$ is computed using chain rule and is of the following form:

$$\boxed{\frac{\partial L(z,y)}{\partial w} = \frac{\partial L(z,y)}{\partial a} \times \frac{\partial a}{\partial z} \times \frac{\partial z}{\partial w}}$$

As a result, the weight is updated as follows:

$$\boxed{w \longleftarrow w - \eta \frac{\partial L(z,y)}{\partial w}}$$

❒ **Updating weights** – In a neural network, weights are updated as follows:

- Step 1: Take a batch of training data.
- Step 2: Perform forward propagation to obtain the corresponding loss.
- Step 3: Backpropagate the loss to get the gradients.
- Step 4: Use the gradients to update the weights of the network.

❒ **Dropout** – Dropout is a technique meant at preventing overfitting the training data by dropping out units in a neural network. In practice, neurons are either dropped with probability $p$ or kept with probability $1-p$.

## 3.2 Convolutional Neural Networks

❒ **Convolutional layer requirement** – By noting $W$ the input volume size, $F$ the size of the convolutional layer neurons, $P$ the amount of zero padding, then the number of neurons $N$ that fit in a given volume is such that:

$$\boxed{N = \frac{W - F + 2P}{S} + 1}$$

❒ **Batch normalization** – It is a step of hyperparameter $\gamma, \beta$ that normalizes the batch $\{x_i\}$. By noting $\mu_B, \sigma_B^2$ the mean and variance of that we want to correct to the batch, it is done as follows:

$$\boxed{x_i \longleftarrow \gamma \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}} + \beta}$$

It is usually done after a fully connected/convolutional layer and before a non-linearity layer and aims at allowing higher learning rates and reducing the strong dependence on initialization.

## 3.3 Recurrent Neural Networks

❒ **Types of gates** – Here are the different types of gates that we encounter in a typical recurrent neural network:

| Input gate | Forget gate | Output gate | Gate |
|---|---|---|---|
| Write to cell or not? | Erase a cell or not? | Reveal a cell or not? | How much writing? |

❒ **LSTM** – A long short-term memory (LSTM) network is a type of RNN model that avoids the vanishing gradient problem by adding 'forget' gates.

### 3.4 Reinforcement Learning and Control

The goal of reinforcement learning is for an agent to learn how to evolve in an environment.

❒ **Markov decision processes** – A Markov decision process (MDP) is a 5-tuple $(S,A,\{P_{sa}\},\gamma,R)$ where:

- $\mathcal{S}$ is the set of states
- $\mathcal{A}$ is the set of actions
- $\{P_{sa}\}$ are the state transition probabilities for $s \in \mathcal{S}$ and $a \in \mathcal{A}$
- $\gamma \in [0,1[$ is the discount factor
- $R : \mathcal{S} \times \mathcal{A} \longrightarrow \mathbb{R}$ or $R : \mathcal{S} \longrightarrow \mathbb{R}$ is the reward function that the algorithm wants to maximize

❒ **Policy** – A policy $\pi$ is a function $\pi : \mathcal{S} \longrightarrow \mathcal{A}$ that maps states to actions.

*Remark: we say that we execute a given policy $\pi$ if given a state $s$ we take the action $a = \pi(s)$.*

❒ **Value function** – For a given policy $\pi$ and a given state $s$, we define the value function $V^\pi$ as follows:

$$\boxed{V^\pi(s) = E\Big[R(s_0) + \gamma R(s_1) + \gamma^2 R(s_2) + ...|s_0 = s,\pi\Big]}$$

❒ **Bellman equation** – The optimal Bellman equations characterizes the value function $V^{\pi^*}$ of the optimal policy $\pi^*$:

$$\boxed{V^{\pi^*}(s) = R(s) + \max_{a\in\mathcal{A}} \gamma \sum_{s'\in S} P_{sa}(s')V^{\pi^*}(s')}$$

*Remark: we note that the optimal policy $\pi^*$ for a given state $s$ is such that:*

$$\boxed{\pi^*(s) = \underset{a\in\mathcal{A}}{\text{argmax}} \sum_{s'\in\mathcal{S}} P_{sa}(s')V^*(s')}$$

❒ **Value iteration algorithm** – The value iteration algorithm is in two steps:

- We initialize the value:

$$\boxed{V_0(s) = 0}$$

- We iterate the value based on the values before:

$$\boxed{V_{i+1}(s) = R(s) + \max_{a\in\mathcal{A}} \left[\sum_{s'\in\mathcal{S}} \gamma P_{sa}(s')V_i(s')\right]}$$

❒ **Maximum likelihood estimate** – The maximum likelihood estimates for the state transition probabilities are as follows:

$$\boxed{P_{sa}(s') = \frac{\#\text{times took action } a \text{ in state } s \text{ and got to } s'}{\#\text{times took action } a \text{ in state } s}}$$

❒ **Q-learning** – $Q$-learning is a model-free estimation of $Q$, which is done as follows:

$$\boxed{Q(s,a) \leftarrow Q(s,a) + \alpha\Big[R(s,a,s') + \gamma \max_{a'} Q(s',a') - Q(s,a)\Big]}$$

# 4 Machine Learning Tips and Tricks

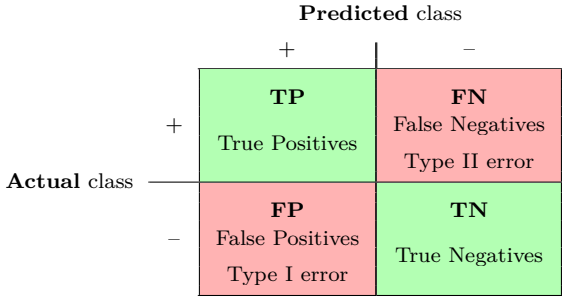
## 4.1 Metrics

Given a set of data points $\{x^{(1)}, ..., x^{(m)}\}$, where each $x^{(i)}$ has $n$ features, associated to a set of outcomes $\{y^{(1)}, ..., y^{(m)}\}$, we want to assess a given classifier that learns how to predict $y$ from $x$.

### 4.1.1 Classification

In a context of a binary classification, here are the main metrics that are important to track to assess the performance of the model.

❒ **Confusion matrix** – The confusion matrix is used to have a more complete picture when assessing the performance of a model. It is defined as follows:



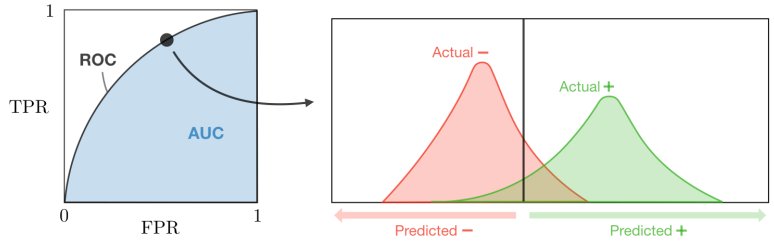

❒ **Main metrics** – The following metrics are commonly used to assess the performance of classification models:

| Metric | Formula | Interpretation |
|---|---|---|
| Accuracy | $\frac{\text{TP}+\text{TN}}{\text{TP}+\text{TN}+\text{FP}+\text{FN}}$ | Overall performance of model |
| Precision | $\frac{\text{TP}}{\text{TP}+\text{FP}}$ | How accurate the positive predictions are |
| Recall<br>Sensitivity | $\frac{\text{TP}}{\text{TP}+\text{FN}}$ | Coverage of actual positive sample |
| Specificity | $\frac{\text{TN}}{\text{TN}+\text{FP}}$ | Coverage of actual negative sample |
| F1 score | $\frac{2\text{TP}}{2\text{TP}+\text{FP}+\text{FN}}$ | Hybrid metric useful for unbalanced classes |

❒ **ROC** – The receiver operating curve, also noted ROC, is the plot of TPR versus FPR by varying the threshold. These metrics are are summed up in the table below:

| Metric | Formula | Equivalent |
|---|---|---|
| True Positive Rate<br>TPR | $\frac{\text{TP}}{\text{TP}+\text{FN}}$ | Recall, sensitivity |
| False Positive Rate<br>FPR | $\frac{\text{FP}}{\text{TN}+\text{FP}}$ | 1-specificity |

❒ **AUC** – The area under the receiving operating curve, also noted AUC or AUROC, is the area below the ROC as shown in the following figure:




### 4.1.2 Regression

❒ **Basic metrics** – Given a regression model $f$, the following metrics are commonly used to assess the performance of the model:

| Total sum of squares | Explained sum of squares | Residual sum of squares |
|---|---|---|
| $\text{SS}_{\text{tot}} = \sum_{i=1}^{m}(y_i - \overline{y})^2$ | $\text{SS}_{\text{reg}} = \sum_{i=1}^{m}(f(x_i) - \overline{y})^2$ | $\text{SS}_{\text{res}} = \sum_{i=1}^{m}(y_i - f(x_i))^2$ |

❒ **Coefficient of determination** – The coefficient of determination, often noted $R^2$ or $r^2$, provides a measure of how well the observed outcomes are replicated by the model and is defined as follows:

$$\boxed{R^2 = 1 - \frac{\text{SS}_{\text{res}}}{\text{SS}_{\text{tot}}}}$$

❒ **Main metrics** – The following metrics are commonly used to assess the performance of regression models, by taking into account the number of variables $n$ that they take into consideration:

| Mallow's Cp | AIC | BIC | Adjusted $R^2$ |
|---|---|---|---|
| $\frac{\text{SS}_{\text{res}} + 2(n+1)\widehat{\sigma}^2}{m}$ | $2\big[(n+2) - \log(L)\big]$ | $\log(m)(n+2) - 2\log(L)$ | $1 - \frac{(1-R^2)(m-1)}{m-n-1}$ |

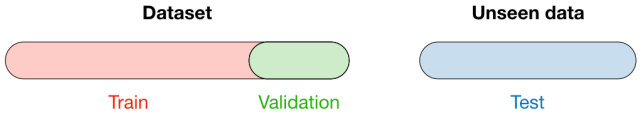
where $L$ is the likelihood and $\widehat{\sigma}^2$ is an estimate of the variance associated with each response.

## 4.2 Model selection

❒ **Vocabulary** – When selecting a model, we distinguish 3 different parts of the data that we have as follows:

| Training set | Validation set | Testing set |
|---|---|---|
| - Model is trained<br>- Usually 80% of the dataset | - Model is assessed<br>- Usually 20% of the dataset<br>- Also called hold-out or development set | - Model gives predictions<br>- Unseen data |

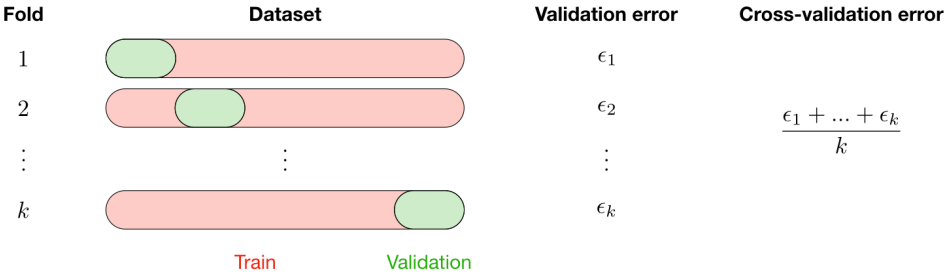
Once the model has been chosen, it is trained on the entire dataset and tested on the unseen test set. These are represented in the figure below:



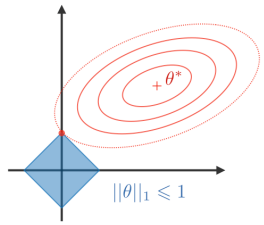
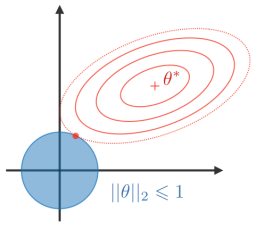
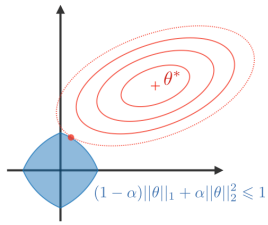

❒ **Cross-validation** – Cross-validation, also noted CV, is a method that is used to select a model that does not rely too much on the initial training set. The different types are summed up in the table below:

| $k$-fold | Leave-$p$-out |
|---|---|
| - Training on $k-1$ folds and assessment on the remaining one<br>- Generally $k = 5$ or 10 | - Training on $n-p$ observations and assessment on the $p$ remaining ones<br>- Case $p = 1$ is called leave-one-out |

The most commonly used method is called $k$-fold cross-validation and splits the training data into $k$ folds to validate the model on one fold while training the model on the $k-1$ other folds, all of this $k$ times. The error is then averaged over the $k$ folds and is named cross-validation error.




❒ **Regularization** – The regularization procedure aims at avoiding the model to overfit the data and thus deals with high variance issues. The following table sums up the different types of commonly used regularization techniques:

| LASSO | Ridge | Elastic Net |
|---|---|---|
| - Shrinks coefficients to 0<br>- Good for variable selection | Makes coefficients smaller | Tradeoff between variable selection and small coefficients |
|  |  |  |
| $... + \lambda\|\|\theta\|\|_1$<br>$\lambda \in \mathbb{R}$ | $... + \lambda\|\|\theta\|\|_2^2$<br>$\lambda \in \mathbb{R}$ | $... + \lambda\left[(1-\alpha)\|\|\theta\|\|_1 + \alpha\|\|\theta\|\|_2^2\right]$<br>$\lambda \in \mathbb{R}$, $\alpha \in [0,1]$ |



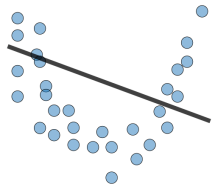
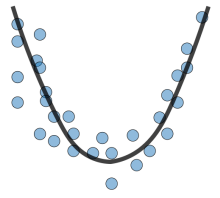
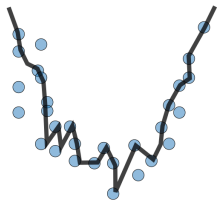
❒ **Model selection** – Train model on training set, then evaluate on the development set, then pick best performance model on the development set, and retrain all of that model on the whole training set.

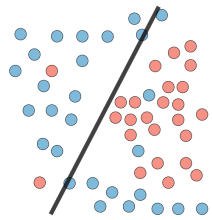
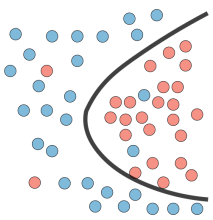
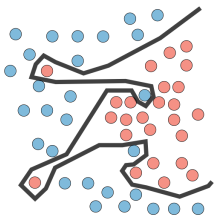
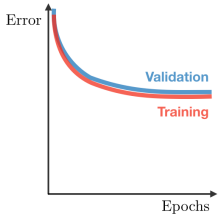
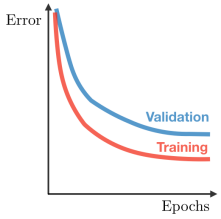
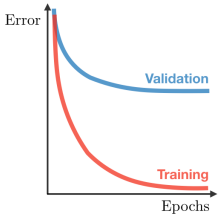
## 4.3 Diagnostics

❒ **Bias** – The bias of a model is the difference between the expected prediction and the correct model that we try to predict for given data points.

❒ **Variance** – The variance of a model is the variability of the model prediction for given data points.

❒ **Bias/variance tradeoff** – The simpler the model, the higher the bias, and the more complex the model, the higher the variance.

| | Underfitting | Just right | Overfitting |
|---|---|---|---|
| **Symptoms** | - High training error<br>- Training error close to test error<br>- High bias | - Training error slightly lower than test error | - Low training error<br>- Training error much lower than test error<br>- High variance |
| **Regression** |  |  |  |

| | | | |
|---|---|---|---|
| **Classification** |  |  |  |
| **Deep learning** |  |  |  |
| **Remedies** | - Complexify model<br>- Add more features<br>- Train longer | | - Regularize<br>- Get more data |

❒ **Error analysis** – Error analysis is analyzing the root cause of the difference in performance between the current and the perfect models.

❒ **Ablative analysis** – Ablative analysis is analyzing the root cause of the difference in performance between the current and the baseline models.

# 5 Refreshers

## 5.1 Probabilities and Statistics

### 5.1.1 Introduction to Probability and Combinatorics

❒ **Sample space** – The set of all possible outcomes of an experiment is known as the sample space of the experiment and is denoted by $S$.

❒ **Event** – Any subset $E$ of the sample space is known as an event. That is, an event is a set consisting of possible outcomes of the experiment. If the outcome of the experiment is contained in $E$, then we say that $E$ has occurred.

❒ **Axioms of probability** – For each event $E$, we denote $P(E)$ as the probability of event $E$ occuring. By noting $E_1$,...,$E_n$ mutually exclusive events, we have the 3 following axioms:

$$(1) \quad \boxed{0 \leqslant P(E) \leqslant 1} \quad (2) \quad \boxed{P(S) = 1} \quad (3) \quad \boxed{P\left(\bigcup_{i=1}^{n} E_i\right) = \sum_{i=1}^{n} P(E_i)}$$

❒ **Permutation** – A permutation is an arrangement of $r$ objects from a pool of $n$ objects, in a given order. The number of such arrangements is given by $P(n,r)$, defined as:

$$\boxed{P(n,r) = \frac{n!}{(n-r)!}}$$

❒ **Combination** – A combination is an arrangement of $r$ objects from a pool of $n$ objects, where the order does not matter. The number of such arrangements is given by $C(n,r)$, defined as:

$$\boxed{C(n,r) = \frac{P(n,r)}{r!} = \frac{n!}{r!(n-r)!}}$$

*Remark: we note that for $0 \leqslant r \leqslant n$, we have $P(n,r) \geqslant C(n,r)$.*

### 5.1.2 Conditional Probability

❒ **Bayes' rule** – For events $A$ and $B$ such that $P(B) > 0$, we have:

$$\boxed{P(A|B) = \frac{P(B|A)P(A)}{P(B)}}$$

*Remark: we have $P(A \cap B) = P(A)P(B|A) = P(A|B)P(B)$.*

❒ **Partition** – Let $\{A_i, i \in [\![1,n]\!]\}$ be such that for all $i$, $A_i \neq \varnothing$. We say that $\{A_i\}$ is a partition if we have:

$$\boxed{\forall i \neq j, A_i \cap A_j = \emptyset \quad \text{and} \quad \bigcup_{i=1}^{n} A_i = S}$$

*Remark: for any event $B$ in the sample space, we have $P(B) = \sum_{i=1}^{n} P(B|A_i)P(A_i)$.*

❒ **Extended form of Bayes' rule** – Let $\{A_i, i \in [\![1,n]\!]\}$ be a partition of the sample space. We have:

$$\boxed{P(A_k|B) = \frac{P(B|A_k)P(A_k)}{\sum_{i=1}^{n} P(B|A_i)P(A_i)}}$$

❒ **Independence** – Two events $A$ and $B$ are independent if and only if we have:

$$\boxed{P(A \cap B) = P(A)P(B)}$$

### 5.1.3 Random Variables

❒ **Random variable** – A random variable, often noted $X$, is a function that maps every element in a sample space to a real line.

❒ **Cumulative distribution function (CDF)** – The cumulative distribution function $F$, which is monotonically non-decreasing and is such that $\lim_{x\to-\infty} F(x) = 0$ and $\lim_{x\to+\infty} F(x) = 1$, is defined as:

$$\boxed{F(x) = P(X \leqslant x)}$$

*Remark: we have $P(a < X \leqslant B) = F(b) - F(a)$.*

❒ **Probability density function (PDF)** – The probability density function $f$ is the probability that $X$ takes on values between two adjacent realizations of the random variable.

❒ **Relationships involving the PDF and CDF** – Here are the important properties to know in the discrete (D) and the continuous (C) cases.

| Case | CDF $F$ | PDF $f$ | Properties of PDF |
|---|---|---|---|
| (D) | $F(x) = \sum_{x_i \leqslant x} P(X = x_i)$ | $f(x_j) = P(X = x_j)$ | $0 \leqslant f(x_j) \leqslant 1$ and $\sum_j f(x_j) = 1$ |
| (C) | $F(x) = \int_{-\infty}^{x} f(y)dy$ | $f(x) = \frac{dF}{dx}$ | $f(x) \geqslant 0$ and $\int_{-\infty}^{+\infty} f(x)dx = 1$ |

❒ **Variance** – The variance of a random variable, often noted $\mathrm{Var}(X)$ or $\sigma^2$, is a measure of the spread of its distribution function. It is determined as follows:

$$\boxed{\mathrm{Var}(X) = E[(X - E[X])^2] = E[X^2] - E[X]^2}$$

❒ **Standard deviation** – The standard deviation of a random variable, often noted $\sigma$, is a measure of the spread of its distribution function which is compatible with the units of the actual random variable. It is determined as follows:

$$\boxed{\sigma = \sqrt{\mathrm{Var}(X)}}$$

❒ **Expectation and Moments of the Distribution** – Here are the expressions of the expected value $E[X]$, generalized expected value $E[g(X)]$, $k^{th}$ moment $E[X^k]$ and characteristic function $\psi(\omega)$ for the discrete and continuous cases:

| Case | $E[X]$ | $E[g(X)]$ | $E[X^k]$ | $\psi(\omega)$ |
|---|---|---|---|---|
| (D) | $\sum_{i=1}^{n} x_i f(x_i)$ | $\sum_{i=1}^{n} g(x_i) f(x_i)$ | $\sum_{i=1}^{n} x_i^k f(x_i)$ | $\sum_{i=1}^{n} f(x_i) e^{i\omega x_i}$ |
| (C) | $\int_{-\infty}^{+\infty} x f(x)dx$ | $\int_{-\infty}^{+\infty} g(x) f(x)dx$ | $\int_{-\infty}^{+\infty} x^k f(x)dx$ | $\int_{-\infty}^{+\infty} f(x) e^{i\omega x} dx$ |

*Remark: we have $e^{i\omega x} = \cos(\omega x) + i\sin(\omega x)$.*

❒ **Revisiting the $k^{th}$ moment** – The $k^{th}$ moment can also be computed with the characteristic function as follows:

$$\boxed{E[X^k] = \frac{1}{i^k}\left[\frac{\partial^k \psi}{\partial \omega^k}\right]_{\omega=0}}$$

❒ **Transformation of random variables** – Let the variables $X$ and $Y$ be linked by some function. By noting $f_X$ and $f_Y$ the distribution function of $X$ and $Y$ respectively, we have:

$$\boxed{f_Y(y) = f_X(x)\left|\frac{dx}{dy}\right|}$$

❒ **Leibniz integral rule** – Let $g$ be a function of $x$ and potentially $c$, and $a, b$ boundaries that may depend on $c$. We have:

$$\boxed{\frac{\partial}{\partial c}\left(\int_a^b g(x)dx\right) = \frac{\partial b}{\partial c} \cdot g(b) - \frac{\partial a}{\partial c} \cdot g(a) + \int_a^b \frac{\partial g}{\partial c}(x)dx}$$

❒ **Chebyshev's inequality** – Let $X$ be a random variable with expected value $\mu$ and standard deviation $\sigma$. For $k, \sigma > 0$, we have the following inequality:

$$\boxed{P(|X - \mu| \geqslant k\sigma) \leqslant \frac{1}{k^2}}$$

### 5.1.4 Jointly Distributed Random Variables

❒ **Conditional density** – The conditional density of $X$ with respect to $Y$, often noted $f_{X|Y}$, is defined as follows:

$$\boxed{f_{X|Y}(x) = \frac{f_{XY}(x,y)}{f_Y(y)}}$$

❒ **Independence** – Two random variables $X$ and $Y$ are said to be independent if we have:

$$\boxed{f_{XY}(x,y) = f_X(x) f_Y(y)}$$

❒ **Marginal density and cumulative distribution** – From the joint density probability function $f_{XY}$, we have:

| Case | Marginal density | Cumulative function |
|---|---|---|
| (D) | $f_X(x_i) = \sum_j f_{XY}(x_i,y_j)$ | $F_{XY}(x,y) = \sum_{x_i \leqslant x} \sum_{y_j \leqslant y} f_{XY}(x_i,y_j)$ |
| (C) | $f_X(x) = \int_{-\infty}^{+\infty} f_{XY}(x,y)dy$ | $F_{XY}(x,y) = \int_{-\infty}^{x}\int_{-\infty}^{y} f_{XY}(x',y')dx'dy'$ |

❒ **Distribution of a sum of independent random variables** – Let $Y = X_1 + ... + X_n$ with $X_1, ..., X_n$ independent. We have:

$$\boxed{\psi_Y(\omega) = \prod_{k=1}^{n} \psi_{X_k}(\omega)}$$

❒ **Covariance** – We define the covariance of two random variables $X$ and $Y$, that we note $\sigma^2_{XY}$ or more commonly $\text{Cov}(X,Y)$, as follows:

$$\boxed{\text{Cov}(X,Y) \triangleq \sigma^2_{XY} = E[(X-\mu_X)(Y-\mu_Y)] = E[XY] - \mu_X\mu_Y}$$

❒ **Correlation** – By noting $\sigma_X, \sigma_Y$ the standard deviations of $X$ and $Y$, we define the correlation between the random variables $X$ and $Y$, noted $\rho_{XY}$, as follows:

$$\boxed{\rho_{XY} = \frac{\sigma^2_{XY}}{\sigma_X\sigma_Y}}$$

*Remarks: For any $X, Y$, we have $\rho_{XY} \in [-1,1]$. If $X$ and $Y$ are independent, then $\rho_{XY} = 0$.*

❒ **Main distributions** – Here are the main distributions to have in mind:

| Type | Distribution | PDF | $\psi(\omega)$ | $E[X]$ | $\text{Var}(X)$ |
|---|---|---|---|---|---|
| (D) | $X \sim \mathcal{B}(n,p)$ Binomial | $P(X=x) = \binom{n}{x} p^x q^{n-x}$ $x \in [\![0,n]\!]$ | $(pe^{i\omega} + q)^n$ | $np$ | $npq$ |
| | $X \sim \text{Po}(\mu)$ Poisson | $P(X=x) = \frac{\mu^x}{x!}e^{-\mu}$ $x \in \mathbb{N}$ | $e^{\mu(e^{i\omega}-1)}$ | $\mu$ | $\mu$ |
| (C) | $X \sim \mathcal{U}(a,b)$ Uniform | $f(x) = \frac{1}{b-a}$ $x \in [a,b]$ | $\frac{e^{i\omega b} - e^{i\omega a}}{(b-a)i\omega}$ | $\frac{a+b}{2}$ | $\frac{(b-a)^2}{12}$ |
| | $X \sim \mathcal{N}(\mu,\sigma)$ Gaussian | $f(x) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2}$ $x \in \mathbb{R}$ | $e^{i\omega\mu - \frac{1}{2}\omega^2\sigma^2}$ | $\mu$ | $\sigma^2$ |
| | $X \sim \text{Exp}(\lambda)$ Exponential | $f(x) = \lambda e^{-\lambda x}$ $x \in \mathbb{R}_+$ | $\frac{1}{1-\frac{i\omega}{\lambda}}$ | $\frac{1}{\lambda}$ | $\frac{1}{\lambda^2}$ |

### 5.1.5 Parameter estimation

❒ **Random sample** – A random sample is a collection of $n$ random variables $X_1, ..., X_n$ that are independent and identically distributed with $X$.

❒ **Estimator** – An estimator $\hat{\theta}$ is a function of the data that is used to infer the value of an unknown parameter $\theta$ in a statistical model.

❒ **Bias** – The bias of an estimator $\hat{\theta}$ is defined as being the difference between the expected value of the distribution of $\hat{\theta}$ and the true value, i.e.:

$$\boxed{\text{Bias}(\hat{\theta}) = E[\hat{\theta}] - \theta}$$

*Remark: an estimator is said to be unbiased when we have $E[\hat{\theta}] = \theta$.*

❒ **Sample mean and variance** – The sample mean and the sample variance of a random sample are used to estimate the true mean $\mu$ and the true variance $\sigma^2$ of a distribution, are noted $\overline{X}$ and $s^2$ respectively, and are such that:

$$\boxed{\overline{X} = \frac{1}{n}\sum_{i=1}^{n} X_i} \quad \text{and} \quad \boxed{s^2 = \hat{\sigma}^2 = \frac{1}{n-1}\sum_{i=1}^{n}(X_i - \overline{X})^2}$$

❒ **Central Limit Theorem** – Let us have a random sample $X_1, ..., X_n$ following a given distribution with mean $\mu$ and variance $\sigma^2$, then we have:

$$\boxed{\overline{X} \underset{n\to+\infty}{\sim} \mathcal{N}\left(\mu, \frac{\sigma}{\sqrt{n}}\right)}$$

## 5.2 Linear Algebra and Calculus

### 5.2.1 General notations

❒ **Vector** – We note $x \in \mathbb{R}^n$ a vector with $n$ entries, where $x_i \in \mathbb{R}$ is the $i^{th}$ entry:

$$x = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix} \in \mathbb{R}^n$$

❒ **Matrix** – We note $A \in \mathbb{R}^{m\times n}$ a matrix with $m$ rows and $n$ columns, where $A_{i,j} \in \mathbb{R}$ is the entry located in the $i^{th}$ row and $j^{th}$ column:

$$A = \begin{pmatrix} A_{1,1} & \cdots & A_{1,n} \\ \vdots & & \vdots \\ A_{m,1} & \cdots & A_{m,n} \end{pmatrix} \in \mathbb{R}^{m\times n}$$

*Remark: the vector $x$ defined above can be viewed as a $n \times 1$ matrix and is more particularly called a column-vector.*

❒ **Identity matrix** – The identity matrix $I \in \mathbb{R}^{n\times n}$ is a square matrix with ones in its diagonal and zero everywhere else:

$$I = \begin{pmatrix} 1 & 0 & \cdots & 0 \\ 0 & \ddots & \ddots & \vdots \\ \vdots & \ddots & \ddots & 0 \\ 0 & \cdots & 0 & 1 \end{pmatrix}$$

*Remark: for all matrices $A \in \mathbb{R}^{n\times n}$, we have $A \times I = I \times A = A$.*

❒ **Diagonal matrix** – A diagonal matrix $D \in \mathbb{R}^{n\times n}$ is a square matrix with nonzero values in its diagonal and zero everywhere else:

$$D = \begin{pmatrix} d_1 & 0 & \cdots & 0 \\ 0 & \ddots & \ddots & \vdots \\ \vdots & \ddots & \ddots & 0 \\ 0 & \cdots & 0 & d_n \end{pmatrix}$$

*Remark: we also note $D$ as $diag(d_1,...,d_n)$.*

### 5.2.2 Matrix operations

❒ **Vector-vector multiplication** – There are two types of vector-vector products:

- inner product: for $x,y \in \mathbb{R}^n$, we have:

$$\boxed{x^T y = \sum_{i=1}^{n} x_i y_i \in \mathbb{R}}$$

- outer product: for $x \in \mathbb{R}^m, y \in \mathbb{R}^n$, we have:

$$\boxed{xy^T = \begin{pmatrix} x_1y_1 & \cdots & x_1y_n \\ \vdots & & \vdots \\ x_my_1 & \cdots & x_my_n \end{pmatrix} \in \mathbb{R}^{m\times n}}$$

❒ **Matrix-vector multiplication** – The product of matrix $A \in \mathbb{R}^{m\times n}$ and vector $x \in \mathbb{R}^n$ is a vector of size $\mathbb{R}^m$, such that:

$$\boxed{Ax = \begin{pmatrix} a_{r,1}^T x \\ \vdots \\ a_{r,m}^T x \end{pmatrix} = \sum_{i=1}^{n} a_{c,i} x_i \in \mathbb{R}^m}$$

where $a_{r,i}^T$ are the vector rows and $a_{c,j}$ are the vector columns of $A$, and $x_i$ are the entries of $x$.

❒ **Matrix-matrix multiplication** – The product of matrices $A \in \mathbb{R}^{m\times n}$ and $B \in \mathbb{R}^{n\times p}$ is a matrix of size $\mathbb{R}^{n\times p}$, such that:

$$\boxed{AB = \begin{pmatrix} a_{r,1}^T b_{c,1} & \cdots & a_{r,1}^T b_{c,p} \\ \vdots & & \vdots \\ a_{r,m}^T b_{c,1} & \cdots & a_{r,m}^T b_{c,p} \end{pmatrix} = \sum_{i=1}^{n} a_{c,i} b_{r,i}^T \in \mathbb{R}^{n\times p}}$$

where $a_{r,i}^T, b_{r,i}^T$ are the vector rows and $a_{c,j}, b_{c,j}$ are the vector columns of $A$ and $B$ respectively.

❒ **Transpose** – The transpose of a matrix $A \in \mathbb{R}^{m\times n}$, noted $A^T$, is such that its entries are flipped:

$$\boxed{\forall i,j, \quad A_{i,j}^T = A_{j,i}}$$

*Remark: for matrices $A,B$, we have $(AB)^T = B^T A^T$.*

❒ **Inverse** – The inverse of an invertible square matrix $A$ is noted $A^{-1}$ and is the only matrix such that:

$$\boxed{AA^{-1} = A^{-1}A = I}$$

*Remark: not all square matrices are invertible. Also, for matrices $A,B$, we have $(AB)^{-1} = B^{-1}A^{-1}$*

❒ **Trace** – The trace of a square matrix $A$, noted $\mathrm{tr}(A)$, is the sum of its diagonal entries:

$$\boxed{\mathrm{tr}(A) = \sum_{i=1}^{n} A_{i,i}}$$

*Remark: for matrices $A,B$, we have $tr(A^T) = tr(A)$ and $tr(AB) = tr(BA)$*

❒ **Determinant** – The determinant of a square matrix $A \in \mathbb{R}^{n\times n}$, noted $|A|$ or $\det(A)$ is expressed recursively in terms of $A_{\backslash i,\backslash j}$, which is the matrix A without its $i^{th}$ row and $j^{th}$ column, as follows:

$$\boxed{\det(A) = |A| = \sum_{j=1}^{n} (-1)^{i+j} A_{i,j} |A_{\backslash i,\backslash j}|}$$

*Remark: $A$ is invertible if and only if $|A| \neq 0$. Also, $|AB| = |A||B|$ and $|A^T| = |A|$.*

### 5.2.3 Matrix properties

❒ **Symmetric decomposition** – A given matrix $A$ can be expressed in terms of its symmetric and antisymmetric parts as follows:

$$\boxed{A = \underbrace{\frac{A + A^T}{2}}_{\text{Symmetric}} + \underbrace{\frac{A - A^T}{2}}_{\text{Antisymmetric}}}$$

❒ **Norm** – A norm is a function $N : V \longrightarrow [0, +\infty[$ where $V$ is a vector space, and such that for all $x,y \in V$, we have:

- $N(x + y) \leqslant N(x) + N(y)$
- $N(ax) = |a|N(x)$ for $a$ scalar
- if $N(x) = 0$, then $x = 0$

For $x \in V$, the most commonly used norms are summed up in the table below:

| Norm | Notation | Definition | Use case |
|---|---|---|---|
| Manhattan, $L^1$ | $\|\|x\|\|_1$ | $\sum_{i=1}^{n} \|x_i\|$ | LASSO regularization |
| Euclidean, $L^2$ | $\|\|x\|\|_2$ | $\sqrt{\sum_{i=1}^{n} x_i^2}$ | Ridge regularization |
| $p$-norm, $L^p$ | $\|\|x\|\|_p$ | $\left(\sum_{i=1}^{n} x_i^p\right)^{\frac{1}{p}}$ | Hölder inequality |
| Infinity, $L^\infty$ | $\|\|x\|\|_\infty$ | $\max_i \|x_i\|$ | Uniform convergence |

❒ **Linearly dependence** – A set of vectors is said to be linearly dependent if one of the vectors in the set can be defined as a linear combination of the others.
*Remark: if no vector can be written this way, then the vectors are said to be linearly independent.*

❒ **Matrix rank** – The rank of a given matrix $A$ is noted $\text{rank}(A)$ and is the dimension of the vector space generated by its columns. This is equivalent to the maximum number of linearly independent columns of $A$.

❒ **Positive semi-definite matrix** – A matrix $A \in \mathbb{R}^{n\times n}$ is positive semi-definite (PSD) and is noted $A \succeq 0$ if we have:

$$\boxed{A = A^T} \quad \text{and} \quad \boxed{\forall x \in \mathbb{R}^n, \quad x^T A x \geqslant 0}$$

*Remark: similarly, a matrix $A$ is said to be positive definite, and is noted $A \succ 0$, if it is a PSD matrix which satisfies for all non-zero vector $x$, $x^T A x > 0$.*

❒ **Eigenvalue, eigenvector** – Given a matrix $A \in \mathbb{R}^{n\times n}$, $\lambda$ is said to be an eigenvalue of $A$ if there exists a vector $z \in \mathbb{R}^n \backslash \{0\}$, called eigenvector, such that we have:

$$\boxed{Az = \lambda z}$$

❒ **Spectral theorem** – Let $A \in \mathbb{R}^{n\times n}$. If $A$ is symmetric, then $A$ is diagonalizable by a real orthogonal matrix $U \in \mathbb{R}^{n\times n}$. By noting $\Lambda = \text{diag}(\lambda_1,...,\lambda_n)$, we have:

$$\boxed{\exists \Lambda \text{ diagonal}, \quad A = U\Lambda U^T}$$

❒ **Singular-value decomposition** – For a given matrix $A$ of dimensions $m \times n$, the singular-value decomposition (SVD) is a factorization technique that guarantees the existence of $U$ $m\times m$ unitary, $\Sigma$ $m \times n$ diagonal and $V$ $n \times n$ unitary matrices, such that:

$$\boxed{A = U\Sigma V^T}$$

### 5.2.4 Matrix calculus

❒ **Gradient** – Let $f : \mathbb{R}^{m\times n} \to \mathbb{R}$ be a function and $A \in \mathbb{R}^{m\times n}$ be a matrix. The gradient of $f$ with respect to $A$ is a $m \times n$ matrix, noted $\nabla_A f(A)$, such that:

$$\boxed{\Big(\nabla_A f(A)\Big)_{i,j} = \frac{\partial f(A)}{\partial A_{i,j}}}$$

*Remark: the gradient of $f$ is only defined when $f$ is a function that returns a scalar.*

❒ **Hessian** – Let $f : \mathbb{R}^n \to \mathbb{R}$ be a function and $x \in \mathbb{R}^n$ be a vector. The hessian of $f$ with respect to $x$ is a $n \times n$ symmetric matrix, noted $\nabla_x^2 f(x)$, such that:

$$\boxed{\Big(\nabla_x^2 f(x)\Big)_{i,j} = \frac{\partial^2 f(x)}{\partial x_i \partial x_j}}$$

*Remark: the hessian of $f$ is only defined when $f$ is a function that returns a scalar.*

❒ **Gradient operations** – For matrices $A$,$B$,$C$, the following gradient properties are worth having in mind:

$$\boxed{\nabla_A \text{tr}(AB) = B^T} \qquad \boxed{\nabla_{A^T} f(A) = (\nabla_A f(A))^T}$$

$$\boxed{\nabla_A \text{tr}(ABA^TC) = CAB + C^TAB^T} \qquad \boxed{\nabla_A |A| = |A|(A^{-1})^T}$$